

[CS230] Foundations of Neural Networks: Logistic Regression as a Neural Network

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1. Deep Learning Big Picture (Introduction) - *Completed*

Chapter 2. Logistic Regression as a Neural Network (Current Unit)

- 2.1 Overview & Terminology
- 2.2 Neural Structure (The Architecture)
- 2.3 The "Admission Officer" Analogy
- 2.4 Cost Function & Optimization
- 2.5 Implementation (Vectorization)

Chapter 3. Shallow Neural Networks (Next Unit)

지난 시간 복습 및 연결

지난 시간, 우리는 딥러닝이라는 거대한 숲(Big Picture)을 보았습니다. 이제 현미경을 꺼내 들 시간입니다. 숲을 이루는 가장 작은 단위인 **나무(뉴런)** 하나를 완벽하게 해부해 봅시다. 로지스틱 회귀를 단순한 통계 기법이 아닌, '**가장 얇은 신경망(Shallow Neural Network)**'으로 이해하는 것이 딥러닝 마스터의 첫걸음입니다.

1 Unit Overview

□ 핵심 요약

이 단원은 딥러닝 모델의 최소 단위인 '**단일 뉴런**'의 작동 원리를 다룹니다.

- **목표:** 이진 분류 문제를 입력 → 선형 계산 → 활성화 → 출력의 신경망 구조로 재해석합니다.
- **핵심:** 계산 그래프를 통해 순전파(Forward)와 역전파(Backward)의 수학적 흐름을 이해합니다.
- **구현:** 'for-loop' 없이 행렬 연산(Vectorization)을 사용하여 효율적인 코드를 작성합니다.
- **이유:** MSE 대신 **Binary Cross-Entropy**를 비용 함수로 사용하는 이유를 불록(Convex) 최적화 관점에서 배웁니다.

2 Essential Terminology

딥러닝 엔지니어들이 숨 쉬듯 사용하는 용어들입니다.

기호	용어	한 줄 정의
x	입력 (Input)	판단의 근거가 되는 데이터 (예: 이미지 픽셀값)
w	가중치 (Weight)	각 입력 정보의 중요도 (클수록 결과에 큰 영향)
b	편향 (Bias)	입력과 무관한 기본 성향 혹은 임계값
z	선형 결과	가중치와 입력을 곱하고 더한 1차 점수 ($w^T x + b$)
$\sigma(z)$	활성화 함수	점수(z)를 확률($0 \sim 1$)로 변환하는 필터 (Sigmoid)
$\hat{y}$	예측값 (Output)	모델이 추측한 정답 확률 (a 라고도 씀)
L	손실 (Loss)	예측이 틀렸을 때 부과하는 벌점 (하나의 데이터)
J	비용 (Cost)	전체 데이터에 대한 손실의 평균 (전체 성적표)

3 Core Concepts: 뉴런의 해부

3.1 1. 신경망적 구조 (The Architecture)

로지스틱 회귀는 두 단계의 생각 과정을 거치는 단일 뉴런입니다.

Step 1: 선형 결합 (Linear Combination)

입력된 정보들을 중요도(w)에 따라 합산합니다.

$$z = w^T x + b$$

Step 2: 비선형 활성화 (Activation)

계산된 점수(z)는 $-\infty \sim \infty$ 범위를 가집니다. 이를 확률($0 \sim 1$)로 바꾸기 위해 시그모이드 (Sigmoid) 함수를 통과시킵니다.

$$\hat{y} = a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

□ 대학 입학 사정관 비유 (직관적 비유)

이 뉴런을 '간간한 입학 사정관'이라고 상상해 봅시다.

1. **입력 (x):** 학생의 내신 성적(x_1), 수능 점수(x_2), 봉사 시간(x_3).
2. **가중치 (w):** 사정관의 평가 기준. (수능이 중요하면 w_2 가 큼).
3. **편향 (b):** 학교의 관대함. (점수가 낮아도 일단 긍정적으로 보면 $b > 0$).
4. **선형 결합 (z):** $z = (x_1 w_1 + x_2 w_2 + x_3 w_3) + b$. (학생의 총점 계산).
5. **활성화 (σ):** 총점이 1000점이든 -500점이든, 합격 확률은 0%에서 100% 사이여야 합니다. 시그모이드는 이 점수를 확률로 매핑합니다.

3.2 2. 비용 함수 (Cost Function): 왜 MSE가 아닌가?

우리는 모델이 정답을 맞히면 칭찬하고, 틀리면 벌점(Cost)을 줘야 합니다. 선형 회귀에서 쓰던 MSE(평균 제곱 오차)를 쓰면 안 될까요?

MSE 사용 금지 정보 (오해 방지 가이드)

로지스틱 회귀(Sigmoid 포함)에 MSE를 적용하면 비용 함수 그래프가 울퉁불퉁한 계단판 모양(Non-Convex)이 됩니다. 경사 하강법을 쓸 때, 가장 깊은 골짜기(Global Minimum)가 아닌 엉뚱한 웅덩이(Local Optima)에 빠져 학습이 멈출 수 있습니다.

그래서 우리는 **매끄러운 그릇 모양(Convex)**을 보장하는 **로그 손실(Binary Cross-Entropy)**을 사용합니다.

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

- 정답이 1인데 0이라고 예측하면? $-\log(0) = \infty$ (무한대의 벌점!)
- 틀릴수록 기하급수적으로 큰 페널티를 부여하여 빠르게 수정하게 만듭니다.

4 Numerical Example: 손으로 푸는 로지스틱 회귀

수식을 눈으로만 보면 이해되지 않습니다. 숫자를 넣어봅시다.

▣ 야간 자율학습 도망자 잡기 시나리오 (실전 시나리오 & 계산)

상황: 선생님(모델)이 학생의 행동을 보고 '도망($y = 1$)' 같지 예측합니다.

- 입력 x : 가방을 째($x_1 = 1$), 눈치를 봄($x_2 = 5$, 매우 많이 봄).
- 가중치 w : $w_1 = 2.0$ (가방 싸는 건 중요), $w_2 = 0.5$ (눈치는 덜 중요).
- 편향 b : -3.0 (선생님은 기본적으로 학생을 믿음).

1. 순전파 (Forward): 예측하기

선형 점수 계산:

$$z = (1 \times 2.0) + (5 \times 0.5) + (-3.0) = 2.0 + 2.5 - 3.0 = 1.5$$

활성화(확률 변환):

$$a = \frac{1}{1 + e^{-1.5}} \approx \frac{1}{1 + 0.223} \approx 0.817$$

→ 선생님은 이 학생이 도망갈 확률을 **81.7%**로 예측했습니다.

2. 역전파 (Backward): 학습하기

실제 결과: 학생이 도망갔습니다 ($y = 1$). 오차 계산 (Magic Step):

$$dz = a - y = 0.817 - 1 = -0.183$$

이 값은 "내가 0.183만큼 부족하게 예측했구나"라는 직관적인 오차입니다. 이제 w 를 업데이트하기 위해 미분값(Gradient)을 구합니다.

$$dw_1 = x_1 \times dz = 1 \times (-0.183) = -0.183$$

$$dw_2 = x_2 \times dz = 5 \times (-0.183) = -0.915$$

3. 파라미터 업데이트 (경사 하강법)

학습률 $\alpha = 0.1$ 이라면:

$$w_1 \leftarrow 2.0 - 0.1(-0.183) = 2.0183$$

결과: w_1 이 증가했습니다. 즉, "가방을 싸는 행동"이 도망에 더 중요한 단서라고 학습했습니다!

5 Implementation: Vectorization (벡터화)

이제 Python으로 구현합니다. m 개의 데이터를 처리할 때 'for' 루프를 쓰면 느립니다. 'numpy'의 행렬 연산을 써야 합니다.

```

1 import numpy as np
2
3 class LogisticUnit:
4     def __init__(self, input_dim):
5         # 는w (input_dim, 1) 크기의 열 벡터로 초기화
6         # 로지스틱 회귀는 으로0 초기화해도 괜찮습니다 (Deep 은NN 안됨!)
7         self.w = np.zeros((input_dim, 1))
8         self.b = 0.0
9
10    def sigmoid(self, z):
11        return 1 / (1 + np.exp(-z))
12
13    def propagate(self, X, Y):
14        """
15        X: (n_x, m) 행렬 - 개의m 데이터가 열로 나열됨
16        Y: (1, m) 벡터 - 정답 레이블
17        """
18        m = X.shape[1] # 데이터 개수
19
20        # 1. Forward Propagation 한( 번에 개m 계산)
21        # (n_x, 1).T @ (n_x, m) + scalar -> (1, m)
22        Z = np.dot(self.w.T, X) + self.b
23        A = self.sigmoid(Z)
24
25        # 비용 계산 (Binary Cross-Entropy)
26        cost = -1/m * np.sum(Y * np.log(A) + (1-Y) * np.log(1-A))
27
28        # 2. Backward Propagation 핵심(: Chain Rule)
29        # dZ = A - Y 예측값과( 실제값의 차이) -> 이 식 하나로 끝냅니다!
30        dZ = A - Y
31
32        # Gradient 계산 (Vectorized)
33        dw = 1/m * np.dot(X, dZ.T)
34        db = 1/m * np.sum(dZ)
35
36        return {"dw": dw, "db": db}, cost

```

Listing 1: Vectorized Logistic Regression Unit

6 FAQ: 초심자가 자주 묻는 질문

- **Q1. 편향(b)이 왜 필요한가요? 없으면 안 되나요?**
A. 편향이 없으면 결정 경계가 무조건 원점(0,0)을 지나야 합니다. 예를 들어, 아무런 행동을 안 해도($x = 0$) 합격률이 50%가 넘을 수 있는데, b 가 없으면 이를 표현할 수 없습니다. b 는 그래프를 좌우로 움직이는 '유연성'을 줍니다.
- **Q2. 가중치 w 를 0으로 초기화해도 학습이 되나요?**
A. 로지스틱 회귀에서는 YES. 비용 함수가 밥그릇 모양(Convex)이라서 어디서 시작하든 바닥으로 굴러갑니다. 하지만 나중에 배울 다층 신경망에서는 절대 안 됩니다(대칭성 문제).
- **Q3. 학습률(Learning Rate)을 어떻게 정하나요?**
A. 너무 크면 정답을 지나쳐 발산(Overshooting)하고, 너무 작으면 학습이 영원히 걸립니다. 보통 0.01, 0.001 등으로 시작해 비용(Cost) 그래프가 잘 내려가는지 보며 조정합니다.

다음 단계 (Next Step)

축하합니다! 여러분은 딥러닝의 가장 기본 부품인 '뉴런' 하나를 완벽하게 이해하고 구현했습니다.

다음 장 [Chapter 3. Shallow Neural Networks]에서는 이 뉴런들을 옆으로 나란히 배치하고, 뒤로 연결하여 은닉층(**Hidden Layer**)을 만드는 법을 배웁니다. 뉴런 하나로는 단순한 선형 분류만 가능하지만, 뉴런이 모이면 복잡한 비선형 문제도 해결할 수 있습니다.

□ 단원 요약 (Cheat Sheet)

1. 구조: 로지스틱 회귀 $= z = w^T x + b$ (선형) $\rightarrow \sigma(z)$ (비선형 활성화).
2. 학습: **Cross-Entropy** 비용 함수를 최소화하는 방향으로 w, b 를 업데이트.
3. 수학: 역전파의 핵심 미분 값은 $dZ = A - Y$ (예측 - 정답).
4. 구현: 'for-loop' 대신 'np.dot'을 활용한 **Vectorization** 필수.